LAPORAN

WORKSHOP SISTEM INFORMASI WEBSERVER (TIF330805)

SEMESTER 3



Tugas:
Acara 9

NAMA:
SHOFI MAULANA SHOFA

NIM:
E41250377

PROGRAM STUDI DILUAR KAMPUS UTAMA (PSDKU)
TEKNIK INFORMATIKA KAB. NGANJUK
JURUSAN TEKNOLOGI INFORMASI
POLITEKNIK NEGERI JEMBER
TAHUN 2026

A. Judul Praktikum:

OOP MVC Lanjutan: Object, Dependency, Getter dan Setter

B. Tujuan Praktikum

1. Memahami Object Composition.
2. Memahami Dependency Injection.
3. Menggunakan Getter dan Setter.
4. Menerapkan validasi pada Setter.
5. Melakukan refactoring CRUD mahasiswa menggunakan konsep OOP.

C. Dasar Teori

1. Object Composition adalah membuat sebuah class memiliki object dari class lain.
   Contohnya:
   Controller → memiliki Repository → memiliki Database
2. Dependency Injection adalah memberikan object yang dibutuhkan sebuah class melalui constructor.
3. Getter digunakan untuk mengambil nilai atribut.
4. Setter digunakan untuk mengubah nilai atribut dan dapat digunakan untuk melakukan validasi.

D. Studi Kasus

Pada praktikum ini dilakukan refactoring aplikasi mahasiswa dengan menerapkan Object Composition, Dependency Injection, serta Getter dan Setter.

Database dikelola oleh class Database, kemudian object tersebut diberikan kepada MahasiswaRepository. Repository selanjutnya diberikan kepada MahasiswaController.

Struktur hubungan object: Database - MahasiswaRepository - MahasiswaController - View

Class Mahasiswa menggunakan atribut private dan menyediakan Getter serta Setter. Setter juga digunakan untuk melakukan validasi NIM dan nama.

1. Class Mahasiswa
   app/Models/Mahasiswa.php
   Semua atribut dibuat private.
   Getter digunakan untuk mengambil data dan Setter digunakan untuk mengubah data sekaligus melakukan validasi.

```php
<?php

class Mahasiswa
{
    private int $id;
    private string $nim;
    private string $nama;
    private string $prodi;
    private string $status;
    private ?int $dosen_id;

    public function __construct(
        string $nim = '',
        string $nama = '',
        string $prodi = '',
        string $status = 'aktif',
        ?int $dosen_id = null
    ) {
        $this->setNim($nim);
        $this->setNama($nama);
        $this->setProdi($prodi);
        $this->setStatus($status);
        $this->setDosenId($dosen_id);
    }

    public function getId(): int
    {
        return $this->id;
    }

    public function getNim(): string
    {
        return $this->nim;
    }

    public function getNama(): string
    {
        return $this->nama;
    }

    public function getProdi(): string
    {
        return $this->prodi;
    }
```

```php
    public function getStatus(): string
    {
        return $this->status;
    }

    public function getDosenId(): ?int
    {
        return $this->dosen_id;
    }

    public function setId(int $id): void
    {
        $this->id = $id;
    }

    public function setNim(string $nim): void
    {
        if ($nim === '' || !ctype_digit($nim)) {
            throw new InvalidArgumentException(
                'NIM harus berupa angka.'
            );
        }

        $this->nim = $nim;
    }

    public function setNama(string $nama): void
    {
        $nama = trim($nama);

        if ($nama === '') {
            throw new InvalidArgumentException(
                'Nama tidak boleh kosong.'
                );
        }

        $this->nama = $nama;
    }

    public function setProdi(string $prodi): void
    {
        $prodi = trim($prodi);

        if ($prodi === '') {
            throw new InvalidArgumentException(
```

```
                'Prodi tidak boleh kosong.
                ');
        }

        $this->prodi = $prodi;
    }

    public function setStatus(string $status): void
    {
        if (!in_array($status, ['aktif', 'cuti', 'lulus'])) {
            throw new InvalidArgumentException(
                'Status tidak valid.'
            );
        }

        $this->status = $status;
    }

    public function setDosenId(?int $dosen_id): void
    {
        $this->dosen_id = $dosen_id;
    }
}
```

2. Mahasiswa Repository

   app/Repositories/MahasiswaRepository.php

   Repository menerima object Database melalui constructor.

```
<?php

require_once __DIR__ . '/../Core/Database.php';
require_once __DIR__ . '/../Models/Mahasiswa.php';

class MahasiswaRepository
{
    private PDO $db;

    public function __construct(PDO $db)
    {
        $this->db = $db;
    }

    public function all(): array
    {
        $stmt = $this->db->query(
            "SELECT * FROM mahasiswa ORDER BY id DESC"
```

```
        );

        return $stmt->fetchAll();
    }

    public function find(int $id): ?array
    {
        $stmt = $this->db->prepare(
            "SELECT * FROM mahasiswa WHERE id = :id"
        );

        $stmt->execute([
            'id' => $id
        ]);

        $data = $stmt->fetch();

        return $data ?: null;
    }

    public function create(Mahasiswa $mahasiswa): bool
    {
        $stmt = $this->db->prepare("
            INSERT INTO mahasiswa
            (nim, nama, prodi, status, dosen_id)
            VALUES
            (:nim, :nama, :prodi, :status, :dosen_id)
        ");

        return $stmt->execute([
            'nim' => $mahasiswa->getNim(),
            'nama' => $mahasiswa->getNama(),
            'prodi' => $mahasiswa->getProdi(),
            'status' => $mahasiswa->getStatus(),
            'dosen_id' => $mahasiswa->getDosenId()
        ]);
    }

    public function update(int $id, Mahasiswa $mahasiswa): bool
    {
        $stmt = $this->db->prepare("
            UPDATE mahasiswa SET
                nim = :nim,
                nama = :nama,
                prodi = :prodi,
```

```
                status = :status,
                dosen_id = :dosen_id
            WHERE id = :id
        ");

        return $stmt->execute([
            'id' => $id,
            'nim' => $mahasiswa->getNim(),
            'nama' => $mahasiswa->getNama(),
            'prodi' => $mahasiswa->getProdi(),
            'status' => $mahasiswa->getStatus(),
            'dosen_id' => $mahasiswa->getDosenId()
        ]);
    }

    public function delete(int $id): bool
    {
        $stmt = $this->db->prepare(
            "DELETE FROM mahasiswa WHERE id = :id"
        );

        return $stmt->execute([
            'id' => $id
        ]);
    }
}
```

3. Mahasiswa Controller

   app/Controllers/MahasiswaController.php

   Perhatikan bahwa Controller tidak membuat koneksi database langsung.

   Repository diberikan melalui constructor.

```
<?php

require_once __DIR__ . '/../Models/Mahasiswa.php';
require_once __DIR__ . '/../Repositories/MahasiswaRepository.php';

class MahasiswaController
{
    private MahasiswaRepository $repo;

    public function __construct(MahasiswaRepository $repo)
    {
        $this->repo = $repo;
    }
```

```php
public function index(): void
{
    $mahasiswa = $this->repo->all();

    $content = __DIR__ . '/../Views/mahasiswa/index.php';

    require __DIR__ . '/../Views/layouts/main.php';
}

public function create(): void
{
    $content = __DIR__ . '/../Views/mahasiswa/create.php';

    require __DIR__ . '/../Views/layouts/main.php';
}

public function store(): void
{
    try {

        $mahasiswa = new Mahasiswa(
            trim($_POST['nim']),
            trim($_POST['nama']),
            trim($_POST['prodi']),
            $_POST['status'],
            !empty($_POST['dosen_id'])
                ? (int) $_POST['dosen_id']
                : null
        );

        $this->repo->create($mahasiswa);

        header(
            'Location: /minggu5/acara9/public/mahasiswa'
        );

        exit;

    } catch (InvalidArgumentException $e) {

        echo $e->getMessage();
    }
}
```

```php
    public function edit(int $id): void
    {
        $mahasiswa = $this->repo->find($id);

        if (!$mahasiswa) {
            http_response_code(404);
            echo 'Data mahasiswa tidak ditemukan.';
            return;
        }

        $content = __DIR__ . '/../Views/mahasiswa/edit.php';

        require __DIR__ . '/../Views/layouts/main.php';
    }

    public function update(int $id): void
    {
        try {

            $mahasiswa = new Mahasiswa(
                trim($_POST['nim']),
                trim($_POST['nama']),
                trim($_POST['prodi']),
                $_POST['status'],
                !empty($_POST['dosen_id'])
                    ? (int) $_POST['dosen_id']
                    : null
            );

            $this->repo->update($id, $mahasiswa);

            header(
                'Location: /minggu5/acara9/public/mahasiswa'
            );

            exit;

        } catch (InvalidArgumentException $e) {

            echo $e->getMessage();
        }
    }

    public function destroy(int $id): void
    {
```

```
        $this->repo->delete($id);

        header(
            'Location: /minggu5/acara9/public/mahasiswa'
        );

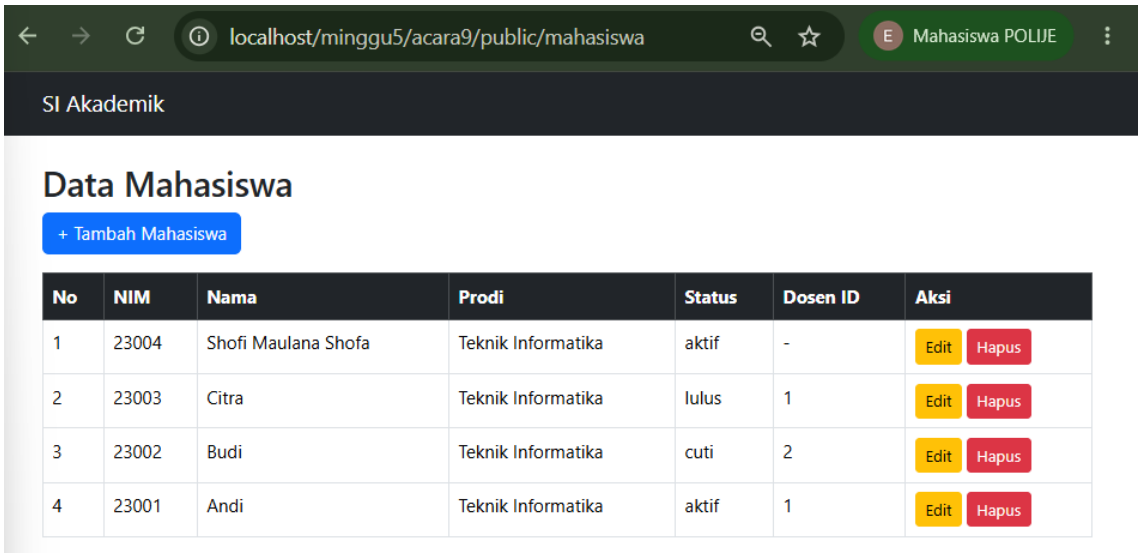
        exit;
    }
}
```

E. Hasil Studi Kasus

1. Uji Read

   http://localhost/minggu5/acara9/public/mahasiswa

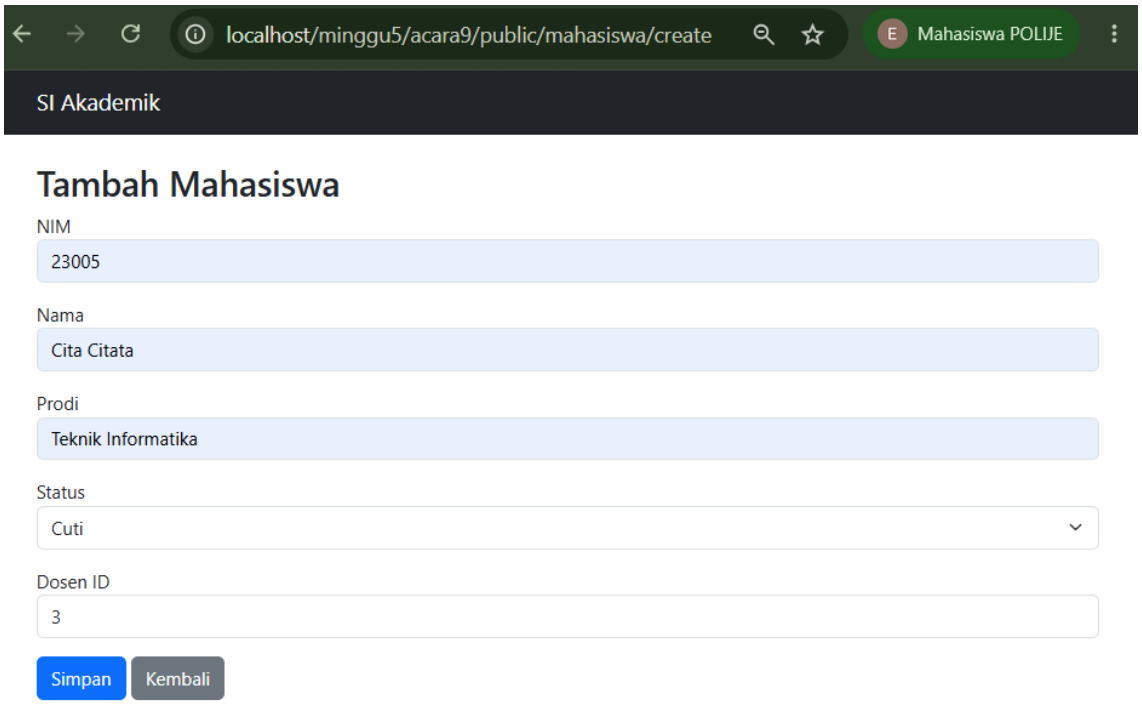
   Data mahasiswa tampil pada halaman.



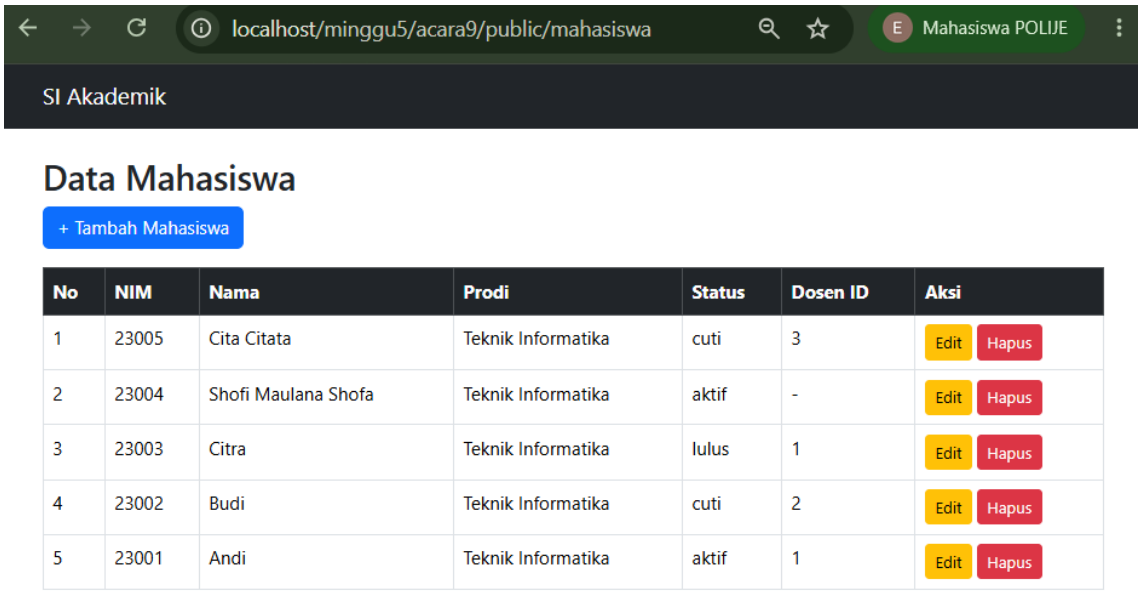
   Hasil: berhasil menampilkan data mahasiswa.

2. Uji Create

   Masukkan:

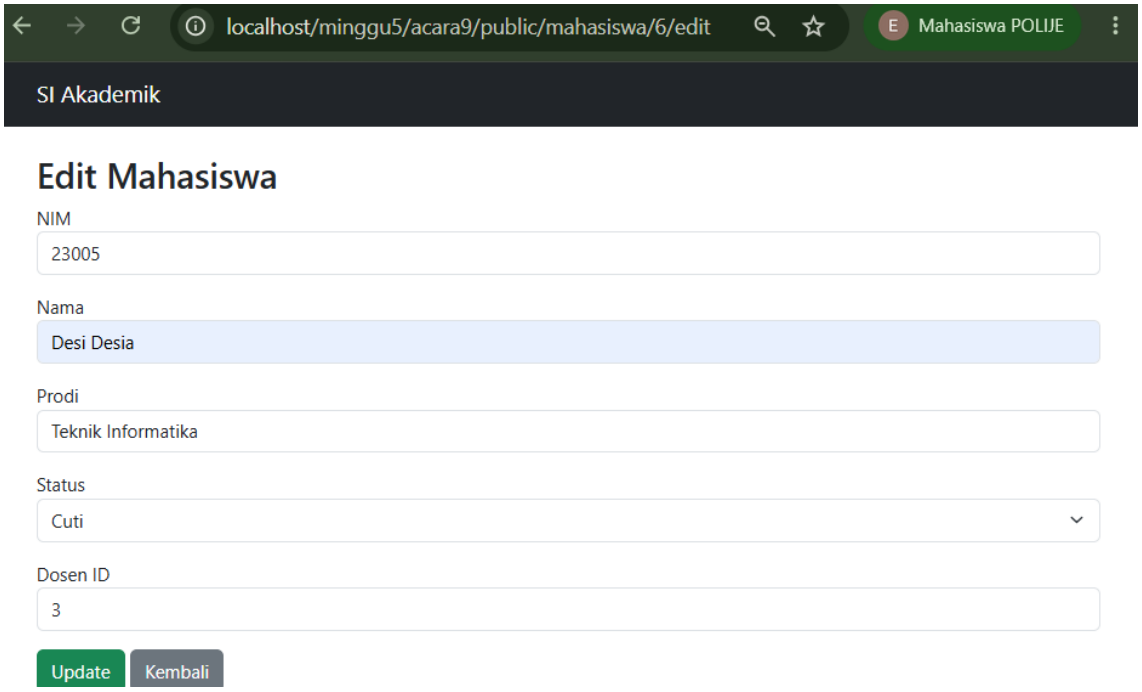


   Hasil: data berhasil masuk ke database.

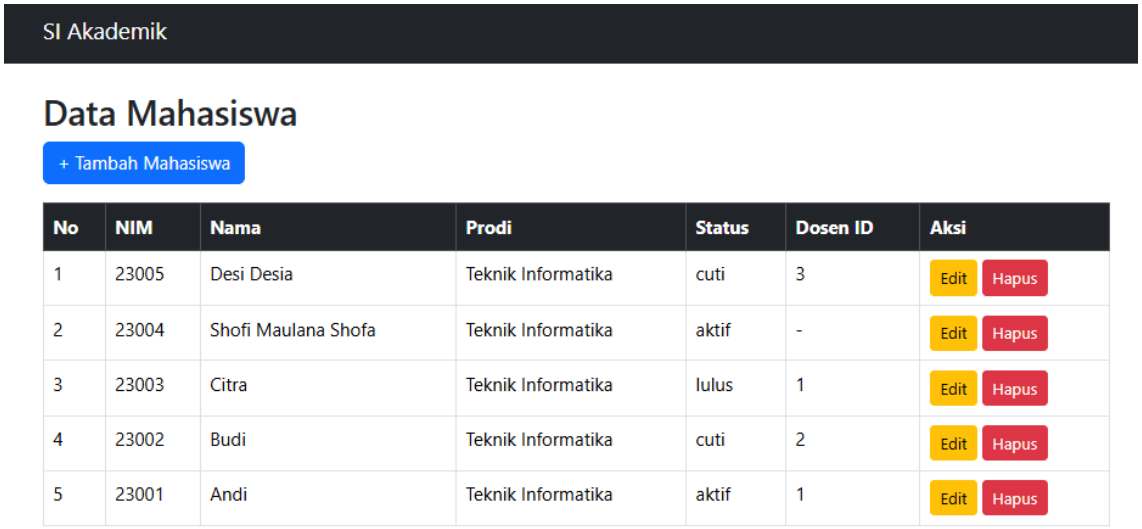


3. Uji Update

   Klik Edit, kemudian ubah nama:



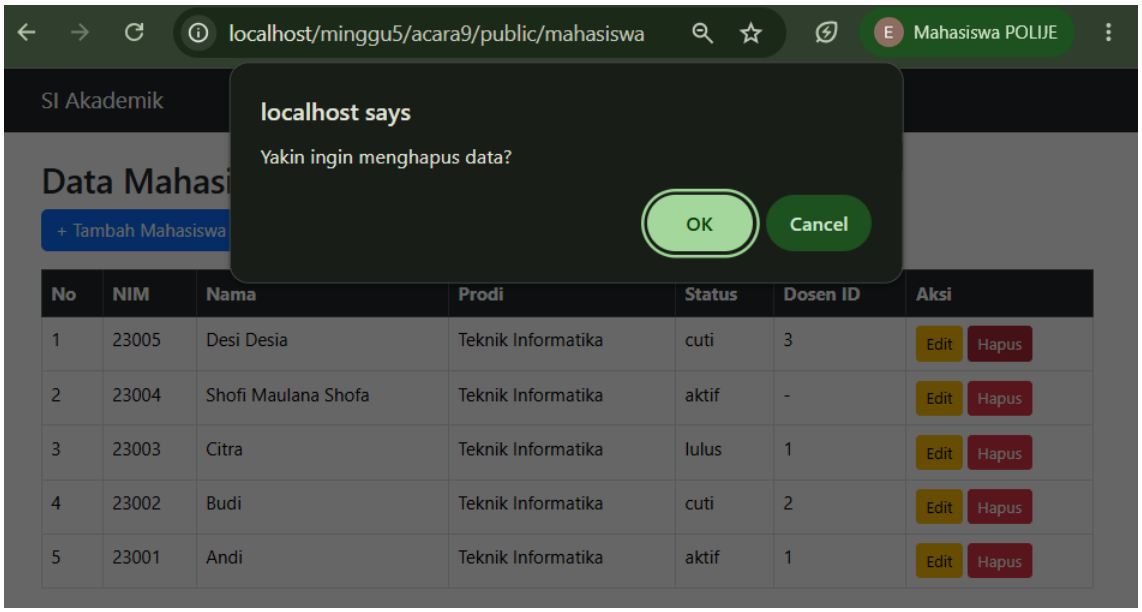
Hasil: data berhasil diperbarui.



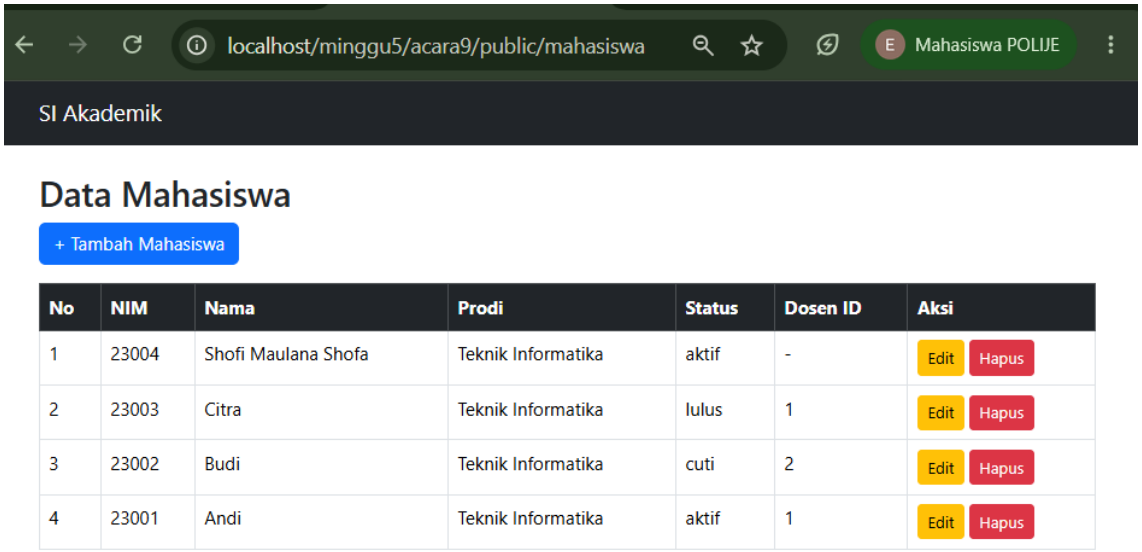
4. Uji Delete

   Klik Hapus.

   Konfirmasi penghapusan.

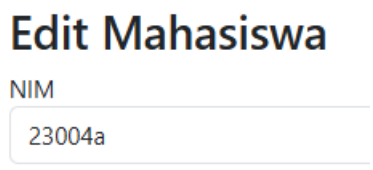


Hasil: data berhasil dihapus.



5. Uji Validasi Setter

   Coba memasukkan NIM:

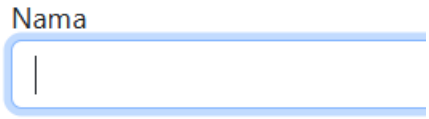


   Setter setNim() akan menolak data karena NIM harus berupa angka.

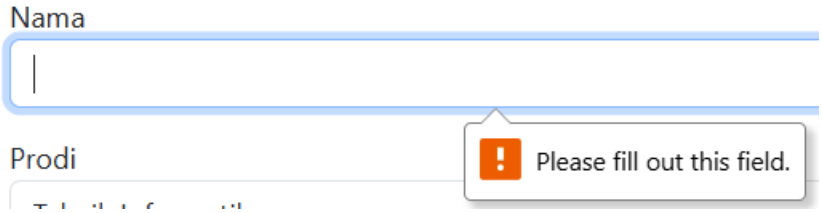


NIM harus berupa angka.

Coba kosongkan nama.



Setter setNama() akan menolak data karena nama tidak boleh kosong.



F. Analisis

Pada Acara 9, kode CRUD diperbaiki dengan konsep OOP. Controller tidak lagi membuat koneksi database secara langsung. Dependency diberikan melalui constructor sehingga kode lebih terstruktur. Getter dan Setter digunakan untuk mengatur akses atribut dan melakukan validasi data.

G. Kesimpulan

Praktikum Acara 9 berhasil menerapkan Object Composition, Dependency Injection, Getter, dan Setter pada aplikasi MVC. Proses CRUD tetap dapat berjalan setelah dilakukan refactoring, serta validasi sederhana berhasil diterapkan pada data mahasiswa.